

Java Backend Architecture

Interview Series

Chapter 10.2

Elasticsearch Architecture

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 10.2 – Elasticsearch Architecture

Introduction

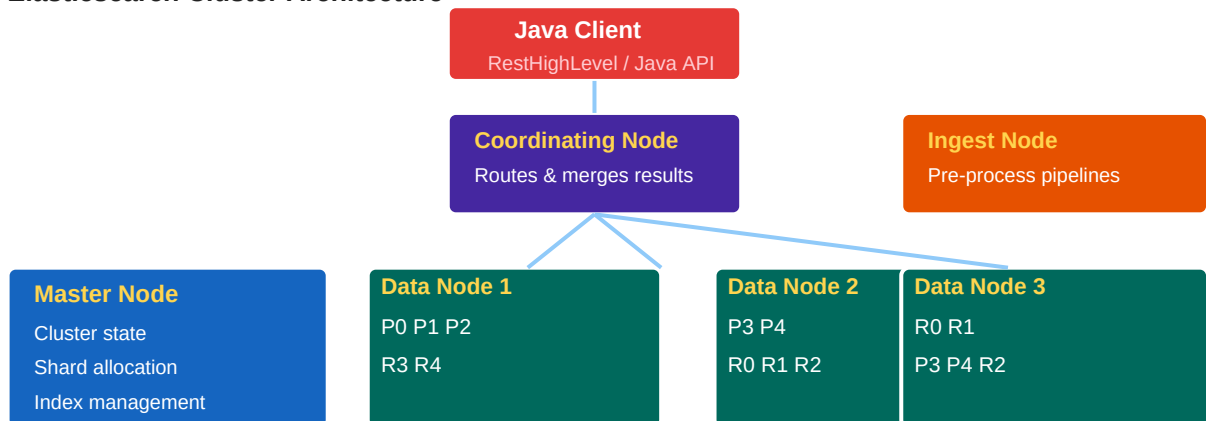
Elasticsearch is a distributed, RESTful search and analytics engine built on Apache Lucene. It scales horizontally by distributing data across **shards** — each a full Lucene index — across multiple **nodes** in a **cluster**. The architecture separates concerns into dedicated node roles: **master** (cluster state), **data** (storage/search), **ingest** (pre-processing), and **coordinating** (routing/merging). Understanding this architecture is critical for designing reliable, scalable Java search systems.

Core Concepts & Definitions

Cluster	Collection of nodes sharing the same cluster.name; single logical search engine.
Node	A running instance of Elasticsearch; plays one or more roles.
Master Node	Manages cluster state: index creation/deletion, shard allocation, node joins/leaves.
Data Node	Stores shards and executes search and indexing operations.
Coordinating Node	Routes client requests to appropriate shards, merges partial results.
Ingest Node	Runs pre-indexing pipelines (processors: grok, set, remove, date, rename, etc.).
Shard	A single Lucene index; unit of distribution. Each index split into N primary shards.
Primary Shard	The authoritative copy of a shard; receives all writes; immovable count after creation.
Replica Shard	Copy of a primary for HA and read scalability; can be re-assigned.
Index	Logical collection of documents with shared mapping; backed by N shards.
Routing	Determines which shard a document is assigned to: $\text{hash}(\text{routing}) \% \text{num_shards}$.
Cluster State	Metadata held by master: mapping, settings, shard allocation, node membership.
Split Brain	Network partition where two master nodes think they are the leader — prevented by quorum.
Quorum	Minimum votes for master election: $(\text{eligible_masters} / 2) + 1$; prevents split brain.
Refresh	Makes in-memory writes searchable by flushing buffer to a new Lucene segment (every 1s by default).
Flush	Persists Lucene segments to disk with fsync + truncates translog.

Cluster Architecture

Elasticsearch Cluster Architecture



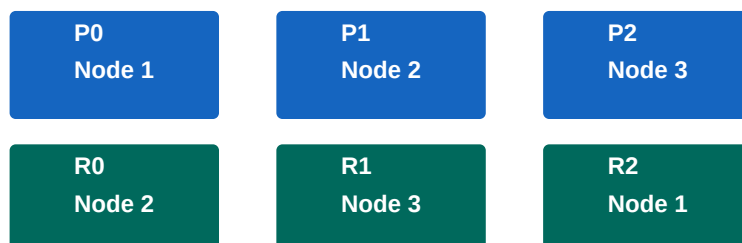
P0=Primary Shard 0, R0=Replica of Shard 0. Primary & replica NEVER on same node.
Master election: Raft-based (ES 7+) via voting configuration.

Dedicated roles: `node.roles=[master]`, `[data]`, `[ingest]`, `[coordinating]` for production clusters

Shard Routing & Query Execution

Shard Routing & Scatter-Gather Query Execution

Index: products (3 primary shards, 1 replica each)



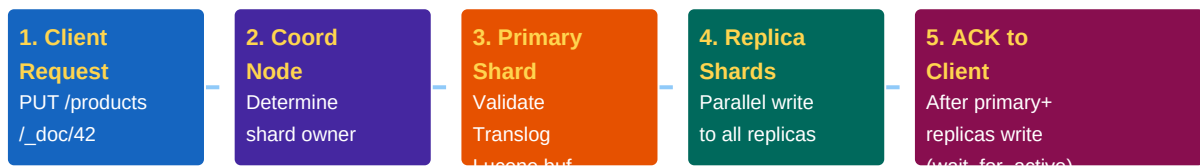
Query Execution:

1. Coordinating node receives query
2. Broadcasts to P0, P1, P2 (or R)
3. Each shard executes locally
4. Returns top-N doc IDs + scores
5. Coord merges, re-ranks globally
6. Fetches `_source` for top results
7. Returns final response to client

Routing formula: `shard = hash(routing_value) % number_of_shards`

Write Path

Elasticsearch Write Path



Refresh (1s): in-memory buffer → new segment (searchable). Flush (30m/translog full): Lucene commit + fsync.
 Translog (WAL): every write appended before Lucene; replayed on crash. durability: request=fsync each write.
 index.number_of_replicas: 1 default. write_consistency: quorum (default), one, all.

Cluster Setup & Index Creation

```
# Create index with explicit shard configuration PUT /products { "settings": {
"number_of_shards": 3, # immutable after creation "number_of_replicas": 1, # mutable; 0 for
single-node dev "refresh_interval": "1s", "index.translog.durability": "request",
"index.translog.flush_threshold_size": "512mb" }, "mappings": { "properties": { "name": {
"type": "text", "analyzer": "standard" }, "category": { "type": "keyword" }, "price": { "type":
"double" }, "created": { "type": "date" } } } } }
```

Node Configuration (elasticsearch.yml)

```
# Dedicated master node node.name: master-1 node.roles: [master] cluster.name: prod-search
network.host: 0.0.0.0 discovery.seed_hosts: ["master-1", "master-2", "master-3"]
cluster.initial_master_nodes: ["master-1", "master-2", "master-3"] # Dedicated data node
node.name: data-1 node.roles: [data] path.data: /var/lib/elasticsearch path.logs:
/var/log/elasticsearch bootstrap.memory_lock: true ES_JAVA_OPTS: "-Xms16g -Xmx16g" #
Coordinating-only node (load balancer) node.name: coord-1 node.roles: [] # empty = coordinating
only # Hot-Warm-Cold architecture node.roles: [data_hot] # fast SSD nodes for recent data
node.roles: [data_warm] # HDDs for older data node.roles: [data_cold] # searchable snapshots
```

Cluster Health & Shard API

```
# Cluster health GET /_cluster/health # status: green (all shards assigned), yellow (replicas
missing), red (primary missing) # Shard allocation GET
/_cat/shards?v&h=index,shard,prirep,state,node # Index stats GET /products/_stats # Node stats
GET /_nodes/stats # Explain unassigned shard GET /_cluster/allocation/explain { "index":
"products", "shard": 0, "primary": false } # Reroute shard manually POST /_cluster/reroute {
"commands": [{ "move": { "index": "products", "shard": 0, "from_node": "data-1", "to_node":
"data-2" } } ] }
```

Index Templates & ILM (Index Lifecycle Management)

```
# Index template for time-series (logs-*) PUT /_index_template/logs_template {
"index_patterns": ["logs-*"], "template": { "settings": { "number_of_shards": 1,
"number_of_replicas": 1, "index.lifecycle.name": "logs_policy",
"index.lifecycle.rollover_alias": "logs" } } } # ILM policy: hot → warm → cold → delete PUT
/_ilm/policy/logs_policy { "policy": { "phases": { "hot": { "actions": { "rollover": {
"max_size": "50gb", "max_age": "1d" } } }, "warm": { "min_age": "7d", "actions": { "shrink": {
"number_of_shards": 1 }, "forcemerge": { "max_num_segments": 1 } } }, "cold": { "min_age":
"30d", "actions": { "freeze": {} } }, "delete": { "min_age": "90d", "actions": { "delete": {} }
} } } }
```

Java REST Client Setup

```
<!-- pom.xml --> <dependency> <groupId>co.elastic.clients</groupId>
<artifactId>elasticsearch-java</artifactId> <version>8.11.0</version> </dependency>

// Elasticsearch Java API Client (8.x) RestClient restClient = RestClient.builder( new
HttpHost("es-host", 9200, "https")) .setHttpClientConfigCallback(builder ->
builder.setDefaultCredentialsProvider( new BasicCredentialsProvider() {{
setCredentials(AuthScope.ANY, new UsernamePasswordCredentials("elastic", "password")); }}))
.build(); ElasticsearchTransport transport = new RestClientTransport( restClient, new
JacksonJsonpMapper()); ElasticsearchClient client = new ElasticsearchClient(transport); //
Index a document IndexResponse resp = client.index(i -> i .index("products") .id("42")
.document(new Product(42L, "Quick Brown Fox Shoes", 49.99))); // Search SearchResponse<Product>
search = client.search(s -> s .index("products") .query(q -> q .match(m ->
m.field("name").query("fox shoes"))), Product.class); search.hits().hits().forEach(hit ->
System.out.println(hit.source() + " score=" + hit.score()));
```

Interview Questions & Answers

Q1. What is an Elasticsearch cluster?

A cluster is a collection of nodes with the same cluster.name that form a single distributed search engine. One node is elected master (manages cluster state); data nodes store shards; coordinating nodes route queries. All nodes expose the REST API; clients can connect to any node.

Q2. What are the different Elasticsearch node roles?

master: manages cluster state (index/shard management). data: stores data, executes queries. ingest: runs pre-indexing processing pipelines. coordinating (empty roles []): routes requests, merges results — acts as smart load balancer. In small clusters one node plays all roles; in production, dedicate roles for stability.

Q3. What is a shard and why does Elasticsearch use them?

A shard is a single Lucene index — the atomic unit of distribution. An index's data is split across N primary shards distributed across data nodes, enabling parallelism and horizontal scaling. num_shards is immutable at index creation; choose based on expected data volume and node count.

Q4. What is the difference between a primary shard and a replica shard?

Primary: authoritative copy; receives all writes; exactly one per shard slot. Replica: copy of primary; provides HA (survives primary failure) and read scalability (queries can be routed to replicas). Primary and replica of same shard are NEVER on the same node.

Q5. What determines which shard a document is stored on?

Default routing: shard = hash(_id) % number_of_shards. Custom routing: pass ?routing=value; shard = hash(routing_value) % number_of_shards. Custom routing co-locates related documents for efficient filtered queries, but risks hot shards on skewed distribution.

Q6. How does Elasticsearch execute a search query?

Scatter-gather: 1) Client sends to any coordinating node. 2) Coord broadcasts to one copy (primary or replica) of each shard. 3) Each shard executes query locally, returns top-N doc_ids and scores. 4) Coord merges, re-ranks to global top-N. 5) Fetches _source for final results (fetch phase). 6) Returns merged response to client.

Q7. What is the cluster state in Elasticsearch?

The cluster state contains: index metadata (mappings, settings, aliases), shard allocation table (which shard on which node), node membership, templates, ILM policies. Maintained by master. Published to all nodes on

change. Critical path: large cluster states slow down master.

Q8. What is split brain in Elasticsearch and how is it prevented?

Split brain: network partition causes two nodes to each think they are master and accept writes independently — results in data divergence. Prevented by `minimum_master_nodes` quorum: $(\text{eligible_masters} / 2) + 1$. ES 7+ uses Raft-based consensus with voting configuration — automatic quorum management.

Q9. What is the difference between yellow and red cluster health?

Green: all primary + replica shards assigned and healthy. Yellow: all primaries assigned, but some replicas unassigned (common in single-node clusters with replicas > 0). Red: at least one primary shard unassigned — index data not fully available; some searches return partial results.

Q10. What is a refresh in Elasticsearch?

Refresh flushes in-memory write buffer to a new Lucene segment, making new documents searchable. Default: every 1 second. `index.refresh_interval`: -1 disables for bulk indexing (re-enable after). Refresh is lightweight (no fsync). After refresh, documents appear in search results.

Q11. What is the difference between refresh and flush?

Refresh: buffer → new segment (searchable); no fsync; lightweight; every 1s. Flush: Lucene commit (fsync all segments to disk) + truncate translog; durable; every 30min or when translog exceeds `flush_threshold_size`. Flush is expensive; don't trigger unnecessarily.

Q12. What is the translog?

Write-ahead log (WAL) for Elasticsearch: every write appended to translog before Lucene. On crash before Lucene commit, translog is replayed to recover operations. `translog.durability`: request = fsync after every write (safe); async = fsync every `sync_interval` (faster, loses up to `sync_interval` data on crash).

Q13. What is an ingest pipeline?

A sequence of processors applied to documents before indexing. Processors: grok (regex parsing), set (add field), remove (delete field), date (parse date string), geoip (enrich with geo from IP), rename, convert, script. Define: `PUT /_ingest/pipeline/my_pipeline`. Use: `PUT /index/_doc/1?pipeline=my_pipeline`.

Q14. What is ILM (Index Lifecycle Management)?

Automates managing time-series indices through phases: hot (active writes/reads), warm (read-only, shrink/forcemerge), cold (frozen/searchable snapshots), delete. Triggered by age or size. Critical for log/metric indices that would otherwise grow unbounded.

Q15. What is a data stream in Elasticsearch?

A data stream is an abstraction over a sequence of time-series backing indices, supporting append-only writes. Write index: current active backing index. Read: queries across all backing indices. Rollover: creates new backing index when hot phase triggers. ILM manages lifecycle. Required for Elastic's Observability stack.

Q16. What is the scroll API and when is it used?

Scroll retrieves large result sets efficiently by maintaining a search context (snapshot) on shards. Like a cursor: `GET /index/_search?scroll=1m` → returns `scroll_id`; `GET /_search/scroll` with `scroll_id` for next page. Not for real-time (stale after scroll window). For new code, prefer `search_after` (stateless, no resource lock).

Q17. What is search_after?

Stateless deep pagination using the sort values of the last returned document as the cursor. `GET /index/_search` with `'search_after: [last_sort_value]'`. No scroll context; consistent across refreshes; no memory overhead. Requires a consistent sort field (e.g., tiebreaker on `_id`).

Q18. What is force merge and when should it be used?

POST `/index/_forcemerge?max_num_segments=1` reduces a shard to a single segment, deleting tombstones and minimizing memory overhead. Use on read-only (warm/cold) indices for better query performance. Never force merge a hot (actively written) index — blocks writes, causes excessive I/O.

Q19. What are aliases in Elasticsearch?

Aliases are virtual index names that can point to one or more indices. Use cases: zero-downtime reindex (switch alias from old to new index atomically), abstract versioning (logs alias → current log index), filter aliases (alias with built-in query filter), write alias (single index for writes). POST `/_aliases` to add/remove atomically.

Q20. What is a hot-warm-cold architecture in Elasticsearch?

Tier data by access frequency using node roles: `data_hot` (NVMe SSDs, recent data), `data_warm` (HDDs, 7-30 day data), `data_cold` (frozen/searchable snapshots, > 30 days). ILM transitions indices between tiers. Reduces cost: expensive fast nodes only hold hot data. Elasticsearch Frozen tier: index partially loaded from S3/GCS snapshots on demand.

Q21. What is the coordinating node's role?

Routes client requests to the correct shards, broadcasts search to all relevant shards, collects and merges partial results, re-ranks globally, and fetches `_source` for top results. Adds CPU/memory overhead. In high-throughput clusters, dedicated coordinating-only nodes (`node.roles: []`) act as smart load balancers.

Q22. What is the Elasticsearch discovery mechanism?

ES 7+ uses a Raft-based consensus algorithm. Nodes discover each other via: `discovery.seed_hosts` (static list), EC2 discovery plugin, Kubernetes discovery plugin. `cluster.initial_master_nodes`: bootstrap list for first cluster formation. Voting configuration: set of master-eligible nodes; quorum = majority.

Q23. What is a snapshot in Elasticsearch?

Point-in-time backup of indices or cluster state to a repository (S3, GCS, Azure Blob, HDFS, shared FS). Incremental: only changed segments backed up. Restore: POST `/_snapshot/repo/snap/_restore`. Searchable snapshots: mount snapshot directly as index without full restore (cold tier). SLM (Snapshot Lifecycle Management) automates scheduled snapshots.

Q24. What is the split_brain risk in even-numbered master-eligible nodes?

With 2 master-eligible nodes: quorum = 2. If both nodes lose contact, neither can elect a master (both need 2 votes but only have 1). Cluster becomes unavailable. Solution: always use odd numbers (3, 5, 7) of master-eligible nodes. Minimum 3 for quorum=2, tolerating 1 failure.

Q25. How does Elasticsearch handle index splitting?

POST `/old-index/_split/new-index?settings.number_of_shards=6` splits an existing index into more shards. Requires original shard count to evenly divide new count. Creates hard links — fast, no data copy. Use when original shard count was too small. Alternative: reindex into new index.

Q26. What is cross-cluster search (CCS)?

Search across multiple Elasticsearch clusters via `cluster:index` notation. Configure remote clusters: PUT `/_cluster/settings` with `persistent.cluster.remote`. Query: GET `/remote1:products/_search`. Federated search: aggregate results from multiple independent clusters. Useful for geo-distributed deployments or separate tenant clusters.

Q27. How do you handle Elasticsearch version upgrades?

Rolling upgrade (minor/patch): upgrade one node at a time; cluster stays available. Full cluster restart (major): disable shard allocation, stop all nodes, upgrade, start, re-enable. Use `/_cluster/settings` to disable rebalancing during upgrade. ES supports queries from one major version behind (e.g., 7.x client works with 8.x cluster with compatibility mode).

Q28. What is the difference between `index.number_of_shards` and `index.number_of_routing_shards`?

`number_of_shards`: actual physical shards (immutable). `number_of_routing_shards`: virtual shards used for routing — allows future splits without changing routing formula. Set equal to a power-of-2 multiple of `number_of_shards`. Enables shard splitting without re-routing.

Q29. What is the segment memory footprint in Elasticsearch?

Each segment requires: FST for term dictionary (loaded into heap), `doc_values` on disk (mmaped), stored fields on disk. Many small segments = high heap usage. Force merge + reduce segment count to lower heap. `index.codec`: `best_compression` for write-heavy indices. Monitor: `GET /_cat/segments?v` for segment count and memory.

Q30. How does Elasticsearch achieve near real-time indexing?

The refresh mechanism: new documents written to an in-memory buffer. Every `refresh_interval` (1s), buffer is flushed to a new Lucene segment — making documents searchable without `fsync`. Documents are visible within ~1 second. For immediate visibility: `PUT /index/_doc/42?refresh=true` (synchronous refresh on write, expensive for high throughput).